

Informe de Incidente de Seguridad -4Geeks Academy-

SARA MARTÍ
05/05/2026

INTRODUCCIÓN

Este análisis forma parte de la primera fase del proyecto final, centrada en la corrección del ataque, la identificación de las vulnerabilidades explotadas y la aplicación de medidas para evitar una posible escalada o repetición del incidente.

Para ello, se lleva a cabo un análisis forense mediante Autopsy, realizado sobre una imagen segmentada de disco .E01, creada con FTK Imager y extraída en frío del servidor Debian comprometido.

El objetivo del análisis es identificar posibles actividades sospechosas, determinar cómo se produjo el compromiso del sistema y recopilar evidencias que permitan comprender el alcance del incidente. A partir de estos hallazgos, se plantearán las medidas necesarias para contener el ataque, corregir las configuraciones inseguras y reforzar la seguridad del servidor.

ALCANCE

El alcance de esta primera fase comprende el análisis forense de la imagen extraída del disco del servidor Debian comprometido, con el objetivo de identificar evidencias del incidente, determinar qué servicios fueron afectados y establecer cómo pudo producirse el acceso no autorizado.

También incluye la revisión de registros del sistema, la búsqueda de archivos sospechosos, modificaciones inusuales, procesos anómalos y posibles indicios de malware o rootkits. Además, medidas de contención y corrección, como el bloqueo del exploit, la detención temporal de servicios comprometidos si fuera necesario, la eliminación de usuarios o puertas traseras no autorizadas y el cierre de puertos innecesarios y la mejora de la configuración de seguridad del servidor.

Finalmente, se documentan las evidencias encontradas y las medidas aplicadas o recomendadas para mitigar el ataque, evitar una posible escalada de privilegios y reducir la probabilidad de que se produzca un incidente similar en un futuro.

ENTORNO DEL ANÁLISIS

El análisis se realiza sobre una imagen forense del disco del servidor Debian comprometido. La adquisición de la evidencia se llevó a cabo mediante FTK Imager, generando una imagen segmentada en formato .E01, que posteriormente fue cargada y analizada con Autopsy.

La revisión se realiza en frío, es decir sobre la imagen del disco y no directamente sobre la máquina original. De esta forma se evita modificar el sistema comprometido durante el análisis y se conserva mejor la integridad de la evidencia.

El sistema analizado corresponde a un servidor Debian utilizado como entorno vulnerable dentro del proyecto. Durante la revisión se identifican servicios relacionados con SSH, FTP, Apache, WordPress y MariaDB, por lo que el análisis se centra principalmente en los registros del sistema, archivos de configuración, directorios web y posibles cambios realizados tras el acceso no autorizado.

```

Created By Exterro® FTK® Imager 4.7.3.81

Case Information:
Acquired using: ADI4.7.3.81
Case Number: PF-4GEEKS-2026
Evidence Number: EV-001
Unique description: Imagen forense del disco Debian comprometido
Examiner: Sara Martí
Notes: Imagen creada desde 0.img para análisis forense en Autopsy para proyecto final de 4Geeks Academy

-----

Information for C:\Users\Usuario\Desktop\forense\DEBI-FINAL\debian_comprometida:

Physical Evidentiary Item (Source) Information:
[Device Info]
Source Type: Logical
[Drive Geometry]
Bytes per Sector: 512
Sector Count: 61.511.680
[Image]
Image Type: Raw (dd)
Source data size: 30035 MB
Sector count: 61511680
[Computed Hashes]
MD5 checksum: 4055af6966bd9de074974ce9e8d9eeca
SHA1 checksum: 975c3f4878a51cedec3228a27d52d8ba3dc23f6f

Image Information:
Acquisition started: Sun May 3 19:38:42 2026
Acquisition finished: Sun May 3 19:41:49 2026
Segment list:
C:\Users\Usuario\Desktop\forense\DEBI-FINAL\debian_comprometida.E01
C:\Users\Usuario\Desktop\forense\DEBI-FINAL\debian_comprometida.E02
C:\Users\Usuario\Desktop\forense\DEBI-FINAL\debian_comprometida.E03
COMPUTED HASH : 4055af6966bd9de074974ce9e8d9eeca
COMPUTED HASH : 975c3f4878a51cedec3228a27d52d8ba3dc23f6f

Image Verification Results:
Verification started: Sun May 3 19:41:49 2026
Verification finished: Sun May 3 19:43:45 2026
MD5 checksum: 4055af6966bd9de074974ce9e8d9eeca : verified
SHA1 checksum: 975c3f4878a51cedec3228a27d52d8ba3dc23f6f : verified

```

Imagen 1: Verificación de hashes de la imagen realizada con FTK Imager.

HERRAMIENTAS UTILIZADAS

HERRAMIENTA	PARA QUÉ SE UTILIZA
FTK Imager	Creación de la imagen forense del disco en formato .E01
Autopsy	Análisis forense de la imagen, revisión de archivos, logs y búsqueda de evidencias.
Keyword Search	Búsqueda de palabras clave relacionadas con SSH, WordPress, usuarios, permisos y servicios.
Chkrootkit	Escáner de detección de rootkits y sniffers.
ClamAV	Escáner de detección de malware.
Hydra	Validación controlada de intentos de fuerza bruta y comprobación de bloqueo mediante Fail2Ban.
Fail2Ban	Protección frente a intentos repetidos de autenticación fallida, especialmente en SSH.

CONSIDERACIÓN SOBRE LAS FECHAS

Durante el análisis se detectan diferencias entre las horas mostradas en algunos registros y fechas visualizadas por Autopsy. En varios logs, como los de Apache, las marcas temporales aparecen con zona horaria -0400, mientras que Autopsy muestra las fechas en horario local CEST.

Por este motivo, para correlacionar correctamente los eventos se tiene en cuenta la diferencia horaria de aproximadamente seis horas entre ambas referencias temporales. Así una entrada registrada como 16:49 -0400 equivale aproximadamente a 22:49 CEST.

```
coreutils.info 2020-07-21 19:00:37 -0400
$ date +'%@%s.%N' # %s and %N are GNU extensions.
@1595372437.692722128
```

HALLAZGOS

A continuación, se describen los hallazgos en orden cronológico, del día 8 de octubre de 2024, según estos hallazgos, la hipótesis principal es que hay una preparación en local y luego un acceso externo por SSH.

INSTALACIÓN Y CONFIGURACIÓN DEL SERVICIO VSFTPD

16:08:57→ El usuario `debian` instala el servicio FTP `vsftpd`. Esto puede servir para permitir transferencia de archivos hacia o desde el servidor.

16:09:38→ Edición de la configuración de FTP, indicando que fue configurado manualmente, esa configuración supone la habilitación del usuario `anonymous`.

16:10:37→ Se reinicia FTP para aplicar los cambios realizados en la configuración.

```
$:(%`C;
SYSLOG_TIMESTAMP=Oct 8 16:08:57
MESSAGE= debian : TTY=pts/1 ; PWD=/home/debian ; USER=root ; COMMAND=/usr/bin/apt install vsftpd
_PID=4687
_CMDLINE=sudo apt install vsftpd
_SOURCE_REALTIME_TIMESTAMP=1728418137451093
);h*;
+;p+;
_SOURCE_REALTIME_TIMESTAMP=1728418137453756
(pW\
+;p+;
SYSLOG_TIMESTAMP=Oct 8 16:09:02
_SOURCE_REALTIME_TIMESTAMP=1728418142525809
+;p+;
SYSLOG_TIMESTAMP=Oct 8 16:09:38
MESSAGE= debian : TTY=pts/1 ; PWD=/home/debian ; USER=root ; COMMAND=/usr/bin/nano /etc/vsftpd.conf
_PID=4886
_CMDLINE=sudo nano /etc/vsftpd.conf
_SOURCE_REALTIME_TIMESTAMP=1728418178275876
```

Imagen 2: Evidencia de instalación y configuración del servicio vsftpd.

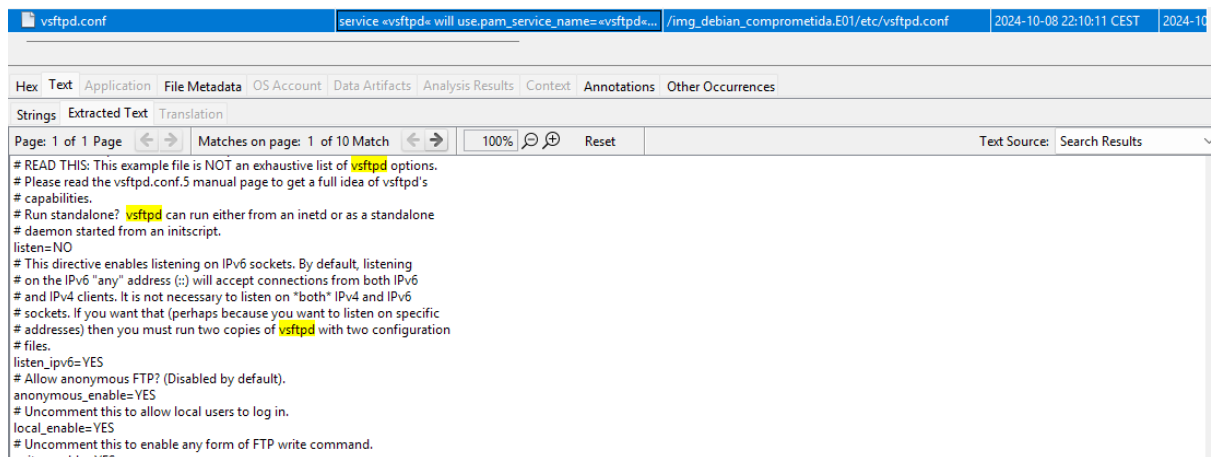


Imagen 3: Evidencia de configuración del servicio vsftpd.

```
I;X;
SYSLOG_TIMESTAMP=Oct 8 16:10:37
MESSAGE= debian : TTY=pts/1 ; PWD=/home/debian ; USER=root ; COMMAND=/usr/bin/systemctl restart vsftpd
_PID=5045
_CMDLINE=sudo systemctl restart vsftpd
_SOURCE_REALTIME_TIMESTAMP=1728418237417757
=:0>;
r@eP
```

Imagen 4: Reinicio del servicio vsftpd para aplicar cambios.

INSTALACIÓN Y CONFIGURACIÓN DEL SERVICIO SSH

16:12:13→ El usuario instala el servidor SSH, habilitando de esa forma una vía de acceso remoto por terminal.

16:12:55→ Configuración de SSH, donde se deja un acceso directo como root a través de este servicio y se permite el acceso con solo autenticación de contraseña, sin necesidad de llave.

16:14:16→ Reinicio del servicio SSH para aplicar cambios en la configuración.

```
v4''U1Z
MESSAGE= debian : TTY=pts/1 ; PWD=/home/debian ; USER=root ; COMMAND=/usr/bin/apt install openssh-server
_PID=5104
_CMDLINE=sudo apt install openssh-server
_SOURCE_REALTIME_TIMESTAMP=172841833817164
~`
```

Imagen 5: Instalación del servicio SSH.

```
"
SYSLOG_TIMESTAMP=Oct 8 16:12:55
MESSAGE= debian : TTY=pts/1 ; PWD=/home/debian ; USER=root ; COMMAND=/usr/bin/nano /etc/ssh/sshd_config
_PID=5157
_CMDLINE=sudo nano /etc/ssh/sshd_config
_SOURCE_REALTIME_TIMESTAMP=1728418375648325
~^^
```

Imagen 6: Configuración del servicio.

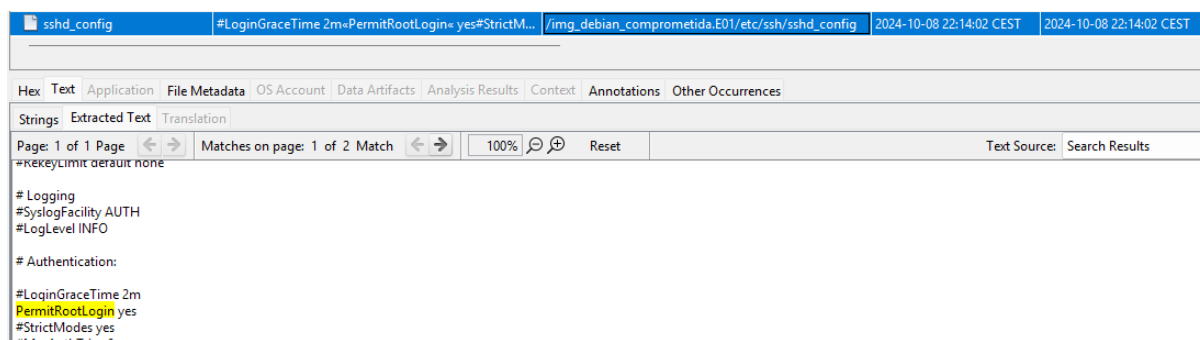


Imagen 7: Habilitación del acceso como root por SSH.

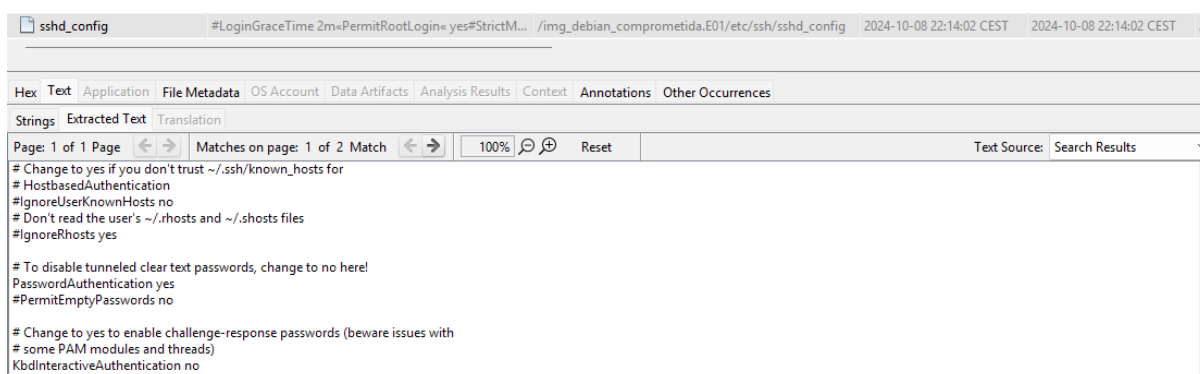


Imagen 8: Habilitación del acceso a SSH con autenticación con contraseña.

```

SYSLOG_TIMESTAMP=Oct 8 16:14:16
MESSAGE=  debian : TTY=pts/1 ; PWD=/home/debian ; USER=root ; COMMAND=/usr/bin/systemctl restart ssh
_PID=5335
_CMDLINE=sudo systemctl restart ssh
_SOURCE_REALTIME_TIMESTAMP=1728418456911886
```

Imagen 9: Reinicio del servicio para aplicar cambios.

INSTALACIÓN DE NET-TOOLS

16:14:59→ Instalación del paquete de net-tools.

16:15:16→ Comprobación de los puertos abiertos y servicios en escucha con netstat, encajando con la hipótesis de una verificación de que FTP y SSH estaban activos.

```

SYSLOG_TIMESTAMP=Oct 8 16:14:59
MESSAGE=  debian : TTY=pts/1 ; PWD=/home/debian ; USER=root ; COMMAND=/usr/bin/apt install net-tools
_PID=5376
_CMDLINE=sudo apt install net-tools
_SOURCE_REALTIME_TIMESTAMP=1728418499491006
```

Imagen 10: Evidencia de la instalación de net-tools.

```

SYSLOG_TIMESTAMP=Oct 8 16:15:16
MESSAGE=  debian : TTY=pts/1 ; PWD=/home/debian ; USER=root ; COMMAND=/usr/bin/netstat -tuln
_PID=5442
_CMDLINE=sudo netstat -tuln
4117R
```

Imagen 11: Ejecución del comando netstat -tuln.

WORDPRESS

16:16:37→ Revisión del directorio web donde se aloja WordPress.

16:17:59→ Concesión de permisos totales sobre todo el directorio web, permitiendo lectura, escritura y ejecución de forma insegura.

16:20:04→ Concesión de permisos totales al archivo wp-config.php, que contiene configuración sensible de WordPress, incluyendo las credenciales de la base de datos.

16:21:23→ Edición de la configuración principal de Apache. En el archivo se observa una configuración demasiado permisiva, especialmente en el bloque <Directory />, donde aparece Require all granted. Esto no implica por sí solo acceso completo a todo el servidor desde la web, pero sí reduce las restricciones de Apache y aumenta el riesgo si se combina con los permisos 777 aplicados previamente sobre WordPress. También se observa el uso de Options Indexes FollowSymLinks, lo que podría facilitar el listado de directorios o el seguimiento de enlaces simbólicos si existieran rutas mal configuradas.

16:47:18→ Reinicio de la red, pudiendo estar relacionado con la aplicación de cambios de conectividad o con la preparación del entorno para acceso remoto.

```
SYSLOG_TIMESTAMP=Oct 8 16:16:37
MESSAGE= debian : TTY=pts/1 ; PWD=/home/debian ; USER=root ; COMMAND=/usr/bin/ls -l /var/www/html
_PID=5480
_CMDLINE=sudo ls -l /var/www/html
_SOURCE_REALTIME_TIMESTAMP=1728418597560748
```

Imagen 12: Revisión de la ruta /var/www/html

```
SYSLOG_TIMESTAMP=Oct 8 16:17:59
MESSAGE= debian : TTY=pts/1 ; PWD=/home/debian ; USER=root ; COMMAND=/usr/bin/chmod -R 777 /var/www/html
_PID=5532
_CMDLINE=sudo chmod -R 777 /var/www/html
SYSLOG_TIMESTAMP=Oct 8 16:20:04
MESSAGE= debian : TTY=pts/1 ; PWD=/home/debian ; USER=root ; COMMAND=/usr/bin/chmod 777 /var/www/html/wp-config.php
_PID=5592
_CMDLINE=sudo chmod 777 /var/www/html/wp-config.php
```

Imagen 13 y 14: Cambio de permisos al directorio web y al archivo wp-config.php

```
SYSLOG_TIMESTAMP=Oct 8 16:21:23
MESSAGE= debian : TTY=pts/1 ; PWD=/home/debian ; USER=root ; COMMAND=/usr/bin/nano /etc/apache2/apache2.conf
lk[k
_PID=5646
_CMDLINE=sudo nano /etc/apache2/apache2.conf
```

Imagen 15: Configuración de Apache.

```
# access here, or in any related virtual host.
<Directory />
    Options Indexes FollowSymLinks
    AllowOverride None
    Require all granted
</Directory>
<Directory /usr/share>
    AllowOverride None
    Options Indexes FollowSymLinks
    Require all granted
</Directory>
<Directory /var/www/>
    Options Indexes FollowSymLinks
    AllowOverride None
    Require all granted
</Directory>
#<Directory /srv/>
#
#     Options Indexes FollowSymLinks
#     AllowOverride None
#     Require all granted
#</Directory>
# AccessFileName: The name of the file to look for in each directory
# for additional configuration directives. See also the AllowOverride
# directive.
AccessFileName .htaccess
# The following lines prevent .htaccess and .htpasswd files from being
# viewed by Web clients.
<FilesMatch "^\.ht">
    Require all denied
</FilesMatch>
# The following directives define some format nicknames for use with
```

Imagen 16: Configuración insegura de Apache.

```
_SOURCE_REALTIME_TIMESTAMP=1728420586
SYSLOG_TIMESTAMP=Oct 8 16:47:18
MESSAGE= debian : TTY=pts/0 ; PWD=/home/debian ; USER=root ; COMMAND=/usr/bin/systemctl restart networking
_PID=2008
_CMDLINE=sudo systemctl restart networking
```

Imagen 17: Reinicio de red.

Después de tocar los permisos de WordPress y Apache, alguien abrió WordPress desde la propia máquina usando Firefox y accedió a localhost/wp-admin. Esto no es un acceso externo, porque aparece 127.0.0.1, pero sí demuestra que se comprobó o accedió a WordPress localmente después de dejarlo con permisos inseguros.

```
127.0.0.1 - - [08/Oct/2024:16:49:46 -0400] "POST /wp-cron.php HTTP/1.1" 200 259 "-" "WordPress/6.6.2; http://localhost"
127.0.0.1 - - [08/Oct/2024:16:49:46 -0400] "GET / HTTP/1.1" 200 3343 "-" "WordPress/6.6.2; http://localhost"
127.0.0.1 - - [08/Oct/2024:16:49:46 -0400] "GET / HTTP/1.1" 200 3343 "-" "WordPress/6.6.2; http://localhost"
127.0.0.1 - - [08/Oct/2024:16:49:46 -0400] "GET / HTTP/1.1" 200 3343 "-" "WordPress/6.6.2; http://localhost"
127.0.0.1 - - [08/Oct/2024:16:49:46 -0400] "POST /wp-cron.php?doing_wp_cron=1728420586.2589499950408935546675 HTTP/1.1" 200 259 "-" "WordPress/6.6.2; http://localhost"
127.0.0.1 - - [08/Oct/2024:16:49:45 -0400] "GET /wp-admin/ HTTP/1.1" 200 17535 "-" "Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0"
127.0.0.1 - - [08/Oct/2024:16:49:46 -0400] "GET /wp-includes/js/thickbox/thickbox.css?ver=6.6.2 HTTP/1.1" 200 1274 "http://localhost/wp-admin/" "Mozilla/5.0 (X11; Linux x86_64; rv:109.0)"
```

Imagen 18: acceso a WordPress desde localhost.

ACCESO POR SSH

17:40:59 → Se confirma un acceso SSH exitoso como root desde la dirección IP 192.168.0.134. Evidencia clara de acceso remoto con privilegios máximos.

```
_SOURCE_REALTIME_TIMESTAMP=1728423659
SYSLOG_TIMESTAMP=Oct 8 17:40:59
MESSAGE=Accepted password for root from 192.168.0.134 port 45623 ssh2
_PID=1650
_CMDLINE="sshd: root [priv]"
_SOURCE_REALTIME_TIMESTAMP=1728423659024171
YMeGD
```

Imagen 19: Evidencia de acceso root por SSH desde la IP 192.168.0.134

ANÁLISIS Y CORRELACIÓN DE EVIDENCIAS

Una vez revisados los registros del sistema, se observa que los hechos no aparecen de forma aislada, sino que siguen una secuencia bastante lógica. Primero se realizan cambios en la máquina desde la cuenta de usuario, después se dejan activos servicios de acceso remoto y finalmente se produce un acceso SSH exitoso como root.

El día 8 de octubre, entre las 16:08 y las 16:21, se identifican varias acciones realizadas con privilegios de administrador mediante sudo. En primer lugar, se instala y configura el FTP vsftpd, lo que podría permitir la transferencia de archivos hacia o desde el servidor. Poco después, también se instala y modifica el servicio SSH mediante el archivo `/etc/ssh/sshd_config`.

La configuración de SSH localizada es relevante, ya que aparecen habilitadas las opciones:

- `PermitRootLogin yes`
- `PasswordAuthentication yes`

Esto significa que el servidor permitía el acceso directo como usuario root mediante contraseña. Esta configuración es insegura, ya que facilita que un atacante pueda acceder directamente con privilegios máximos si consigue la contraseña de root.

Después de modificar el servicio SSH reinicia el servicio con:

- `sudo systemctl restart ssh`

Esto indica que los cambios realizados en la configuración quedaron aplicados.

También se observa la instalación de net-tools y la ejecución de `netstat -tuln`. Esto apunta a una comprobación de los puertos abiertos y de los servicios en escucha, para seguramente verificar que tanto FTP como SSH estaban activos correctamente.

El siguiente paso fue revisar el directorio web donde se aloja WordPress, `/var/www/html` y aplicar permisos muy inseguros:

- `sudo chmod -R 777 /var/www/html`
- `sudo chmod 777 /var/www/html/wp-config.php`

Esta configuración es crítica ya que deja el directorio de WordPress y su archivo principal de configuración con permisos totales. En la práctica, esto facilita que los archivos puedan ser modificados, y en el caso de `wp-config.php`, se expone un archivo sensible que contiene datos importantes de configuración, como credenciales de base de datos.

Posteriormente, hubo un acceso al archivo de configuración principal de Apache:

- `sudo nano /etc/apache2/apache2.conf`

En este archivo se puede observar una configuración demasiado permisiva e insegura, especialmente en el bloque `<Directory/>`, donde aparece `Require all granted`. Aunque esto no significa que automáticamente se pueda acceder a todo el servidor desde la web, sí reduce las restricciones de Apache y aumenta el riesgo si se combina con los permisos 777 aplicados previamente sobre WordPress.

Minutos después, los logs de Apache muestran que el servicio se reinicia y vuelve a funcionar correctamente, probablemente para aplicar los cambios realizados en esta configuración.

Poco más tarde, a las 16:49, los registros de Apache muestran accesos desde 127.0.0.1 hacia WordPress, concretamente a /wp-admin/. Al tratarse de 127.0.0.1, se trata de una interacción desde la propia máquina. Indicando que alguien accedió localmente al panel de administración de WordPress después de haber modificado permisos y configuración del servidor web.

Finalmente, a las 17:50:59, se registra un acceso SSH exitoso como root desde la IP 192.168.0.134:

- Accepted password for root from 192.168.0.134 port 45623 ssh2

Este evento confirma que hubo una conexión remota al servidor con privilegios máximos. Además, encaja con los cambios previos realizados en SSH, ya que el servidor permitía acceso directo como root mediante contraseña.

En conjunto, la correlación de evidencias permite identificar dos fases claras. Primero, una fase de preparación del servidor, donde se instalan servicios remotos, se modifican configuraciones y se debilita la seguridad de WordPress. Después, una fase de acceso remoto, donde se produce una autenticación correcta como root por SSH desde una IP de la red local.

No se puede asegurar que fueran dos personas distintas, pero sí se observa una separación clara entre la preparación previa realizada desde la cuenta debian y el acceso remoto posterior como root. Esta secuencia apunta a que el servidor fue preparado deliberadamente para facilitar el acceso remoto y la posible manipulación del entorno web.

ARCHIVOS SOSPECHOSOS, PROCESOS Y MODIFICACIONES INUSUALES

Durante el análisis se revisaron archivos de configuración, registros del sistema y directorios relacionados con los servicios afectados. No se pudo confirmar la presencia de ningún archivo malicioso concreto en esta fase del análisis, pero sí se identificaron modificaciones inusuales y configuraciones inseguras que facilitaron el compromiso del servidor.

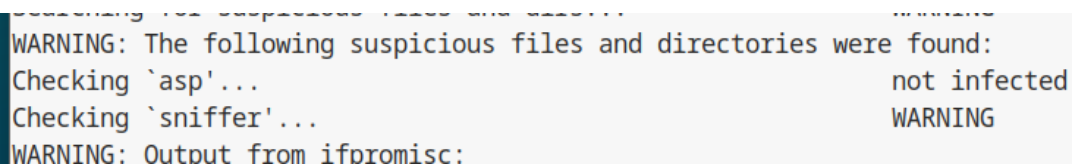
Entre las modificaciones más relevantes se encuentran las ya mencionadas en los apartados previos:

- Instalación y configuración de vsftpd, con la habilitación del usuario Anonymous.
- Instalación y configuración de SSH, con la habilitación de acceso como root y con autenticación por contraseña.
- Modificación de permisos del directorio /var/www/html
- Modificación de permisos del archivo wp-config.php

ESCANEEO DE ROOTKITS O MALWARE

Además del análisis forense en Autopsy, se realiza un escaneo del sistema en busca de rootkits con la herramienta chkrootkit con el objetivo de detectar rootkits o sniffers.

Durante el análisis, chkrootkit generó una advertencia en la comprobación relacionada con posible sniffer:



```
WARNING: The following suspicious files and directories were found:
Checking `asp'... not infected
Checking `sniffer'... WARNING
WARNING: Output from ifpromisc:
```

Imagen 20: Advertencia de chkrootkit.

Se realizó una comprobación con `ip -d link show` y aquí se pudo observar que la interfaz principal no presentaba la bandera PROMISC y mostraba promiscuity 0, por lo que no se confirmó la existencia de un sniffer activo.

A continuación, se realizó un escaneo completo con ClamAV, analizando 29.753 directorios y 154.285 archivos. El resultado fue Infected files: 0, por lo que no se detectó malware conocido. El análisis también registró 632 errores, probablemente relacionados con archivos no accesibles, permisos o rutas especiales del sistema. Estos errores no indican infección.

```
debian@debian:~$ cat clamav_resultado_completo.txt

----- SCAN SUMMARY -----
Known viruses: 3627854
Engine version: 1.4.3
Scanned directories: 29573
Scanned files: 154285
Infected files: 0
Total errors: 632
Data scanned: 8090.91 MB
Data read: 6170.82 MB (ratio 1.31:1)
Time: 2409.678 sec (40 m 9 s)
Start Date: 2026:05:09 12:19:56
End Date: 2026:05:09 13:00:06
debian@debian:~$
```

Imagen 21: Resultado del escaneo completo del servidor mediante ClamAV.

MEDIDAS DE CONTENCIÓN, REVERSIÓN DE CAMBIOS DEL ATAQUE Y CORRECCIÓN

Para contener el incidente, se detuvieron temporalmente los servicios comprometidos o expuestos mientras se revisa y aplica la corrección:

- `sudo systemctl stop vsftpd`
- `sudo systemctl stop ssh`
- `sudo systemctl stop apache2`

No se vuelven a restablecer hasta que las configuraciones corregidas son seguras.

CORRECCIÓN SERVICIO SSH

A continuación, se corrigió la configuración insegura del servicio SSH. Durante el análisis forense se había identificado que el servidor permitía el acceso directo como root y la autenticación mediante contraseña, configuración que facilitó el acceso SSH exitoso detectado desde la IP 192.168.0.134.

Para corregirlo, se modificó el archivo `/etc/ssh/sshd_config`, con la siguiente configuración:

- `PermitRootLogin` no → deshabilitando el inicio de sesión directo como usuario root por SSH.
- `PasswordAuthentication` no → impidiendo el acceso con contraseña y obligando así a utilizar autenticación por clave pública.

Después de aplicar los cambios, se validó la corrección intentando acceder desde kali al servidor Debian mediante SSH, tanto con el usuario debian como con el usuario root.

En ambos casos el acceso fue denegado, confirmando así que la autenticación por contraseña quedó deshabilitada y que ya no es posible acceder por SSH utilizando credenciales débiles.

```
#LoginGraceTime 2m
PermitRootLogin no
#StrictModes yes
#MaxAuthTries 6
#MaxSessions 10

# To disable tunneled clear text passwords, change to no here!
PasswordAuthentication no
#PermitEmptyPasswords no
```

Imagen 22: Corrección de la configuración del servicio SSH

```
(kali㉿kali)-[~/Desktop]
$ ssh debian@10.0.2.26
The authenticity of host '10.0.2.26 (10.0.2.26)' can't be established.
ED25519 key fingerprint is: SHA256:y+azUUsJLjX3WV8+EjMaTB4WybvW7XBLCt7vp3zvLg
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.0.2.26' (ED25519) to the list of known hosts.
debian@10.0.2.26: Permission denied (publickey).

(kali㉿kali)-[~/Desktop]
$ ssh root@10.0.2.26
root@10.0.2.26: Permission denied (publickey).

(kali㉿kali)-[~/Desktop]
$
```

Imagen 22: Validación de acceso SSH denegado para los usuarios debian y root

Como refuerzo adicional frente a intentos de fuerza bruta, se configuró Fail2Ban para proteger el servicio SSH. Para ello se creó el archivo jail.local en /etc/fail2ban/ con una jail específica para sshd. Esta configuración permite que Fail2Ban monitoree los intentos fallidos contra SSH y bloquee temporalmente las direcciones IP que superen el número máximo de intentos permitidos.

```
debian@debian: /etc/fail2ban
File Edit View Search Terminal Help
GNU nano 7.2 jail.local
[sshd]
enabled = true
port = 22
backend = systemd
maxretry = 3
```

Imagen 23: Configuración del archivo jail.local

Después se reinició el servicio y se comprobó el estado del servicio para confirmar que quedaba activo, dando como resultado que estaba activo, quedando claro que la medida queda aplicada correctamente. Y para terminar de comprobar que funcionaba se intentó fuerza bruta con hydra por el servicio SSH para los usuarios debian y root.

```
debian@debian:/etc/fail2ban
File Edit View Search Terminal Help
debian@debian:/$ cd etc
debian@debian:/etc$ cd fail2ban
debian@debian:/etc/fail2ban$ ls
action.d      filter.d      jail.local    paths-debian.conf
fail2ban.conf jail.conf     paths-arch.conf paths-opensuse.conf
fail2ban.d    jail.d        paths-common.conf
debian@debian:/etc/fail2ban$ sudo nano jail.local
debian@debian:/etc/fail2ban$ sudo systemctl restart fail2ban
debian@debian:/etc/fail2ban$ sudo systemctl status fail2ban
● fail2ban.service - Fail2Ban Service
   Loaded: loaded (/lib/systemd/system/fail2ban.service; enabled; preset: ena>
   Active: active (running) since Thu 2026-05-14 11:33:13 EDT; 10s ago
     Docs: man:fail2ban(1)
   Main PID: 66100 (fail2ban-server)
    Tasks: 5 (limit: 2284)
   Memory: 21.3M
      CPU: 334ms
   CGroup: /system.slice/fail2ban.service
           └─66100 /usr/bin/python3 /usr/bin/fail2ban-server -xf start

May 14 11:33:13 debian systemd[1]: Started fail2ban.service - Fail2Ban Service.
May 14 11:33:14 debian fail2ban-server[66100]: 2026-05-14 11:33:14,167 fail2ban>
May 14 11:33:14 debian fail2ban-server[66100]: Server ready
lines 1-14/14 (END)
```

Imagen 24: Servicio Fail2Ban activo tras aplicar la configuración.

```
(kali@kali)~$ hydra -l root -P /usr/share/wordlists/rockyou.txt ssh://10.0.2.26
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these ** ignore la

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-14 12:21:20
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[DATA] max 16 tasks per 1 server, overall 16 tasks, 14344399 login tries (l:1/p:14344399), ~896525 tries per task
[DATA] attacking ssh://10.0.2.26:22/
[ERROR] target ssh://10.0.2.26:22/ does not support password authentication (method reply 4).

(kali@kali)~$ hydra -l root -P /usr/share/wordlists/rockyou.txt ssh://10.0.2.26
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these ** ignore la

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-14 12:21:46
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[DATA] max 16 tasks per 1 server, overall 16 tasks, 14344399 login tries (l:1/p:14344399), ~896525 tries per task
[DATA] attacking ssh://10.0.2.26:22/
[ERROR] target ssh://10.0.2.26:22/ does not support password authentication (method reply 4).
```

Imagen 25: Validación de que Fail2Ban ha funcionado.

Además, se procedió a deshabilitar el servicio SSH, asociado al puerto 22. Esta decisión se tomó porque, según las evidencias analizadas, el servicio fue instalado el mismo día del incidente, por lo que no formaba parte de la configuración habitual del servidor en ese momento.

Si bien en el sistema existen claves y archivos relacionados con SSH, no se ha confirmado que el servicio estuviera habilitado antes del 8 de octubre. Por este motivo, y dado que posteriormente fue utilizado como vía de acceso remoto por el atacante, se considera más seguro mantenerlo desactivado para reducir la superficie de exposición del servidor.

```
debian@debian: ~  
File Edit View Search Terminal Help  
debian@debian:~$ sudo systemctl stop ssh  
debian@debian:~$ sudo systemctl disable ssh  
Synchronizing state of ssh.service with SysV service script with /lib/systemd/sy  
stemd-sysv-install.  
Executing: /lib/systemd/systemd-sysv-install disable ssh  
Removed "/etc/systemd/system/multi-user.target.wants/ssh.service".  
Removed "/etc/systemd/system/sshd.service".  
debian@debian:~$ sudo systemctl status ssh  
○ ssh.service - OpenBSD Secure Shell server  
   Loaded: loaded (/lib/systemd/system/ssh.service; disabled; preset: enabled)  
   Active: inactive (dead)  
     Docs: man:sshd(8)  
           man:sshd_config(5)  
  
May 16 01:12:48 debian systemd[1]: Starting ssh.service - OpenBSD Secure Shell >  
May 16 01:12:48 debian sshd[606]: Server listening on 0.0.0.0 port 22.  
May 16 01:12:48 debian systemd[1]: Started ssh.service - OpenBSD Secure Shell s>  
May 16 01:12:48 debian sshd[606]: Server listening on :: port 22.  
May 16 01:39:26 debian sshd[606]: Received signal 15; terminating.  
May 16 01:39:26 debian systemd[1]: Stopping ssh.service - OpenBSD Secure Shell >  
May 16 01:39:26 debian systemd[1]: ssh.service: Deactivated successfully.  
May 16 01:39:26 debian systemd[1]: Stopped ssh.service - OpenBSD Secure Shell s>  
lines 1-14/14 (END)
```

```
(kali@kali)~$ nmap -p- --open 10.0.2.26  
Starting Nmap 7.98 ( https://nmap.org ) at 2026-05-16 01:44 -0400  
Nmap scan report for 10.0.2.26  
Host is up (0.00014s latency).  
Not shown: 65533 closed tcp ports (reset)  
PORT      STATE SERVICE  
21/tcp    open  ftp  
80/tcp    open  http  
MAC Address: 08:00:27:35:4B:E3 (Oracle VirtualBox virtual NIC)  
Nmap done: 1 IP address (1 host up) scanned in 7.41 seconds
```

Imagen 26: Comprobación de que el puerto 22 ya no está disponible.

Finalmente, como medida de contención adicional, se bloqueó la dirección IP 192.168.0.134, identificada en los registros como origen del acceso SSH exitoso como root. Aunque los servicios expuestos estaban detenidos durante la corrección, se aplicó el bloqueo a todas las conexiones entrantes desde esa IP como medida preventiva antes de volver a levantar los servicios corregidos.

Iptables no estaba disponible inicialmente en el sistema, por lo que se instaló y después se añadió la regla de bloqueo, verificando después la regla aplicada. El resultado mostró una regla DROP para la IP 192.168.0.134, confirmando que cualquier conexión entrante desde esa dirección quedaba bloqueada.


```

debian@debian:/$ sudo iptables -A INPUT -s 192.168.0.134 -j DROP
debian@debian:/$ sudo iptables -L INPUT -n --line-numbers
Chain INPUT (policy ACCEPT)
num target      prot opt source                destination
1    DROP          0    --  192.168.0.134          0.0.0.0/0
debian@debian:/$ █

```

Imagen 27: Iptables configurado para el bloqueo de la IP 192.168.0.134

Con estas medidas se corrige la configuración insegura de SSH, se reduce el riesgo de ataques de fuerza bruta y se bloquea preventivamente la IP identificada como origen del acceso remoto no autorizado.

CORRECCIÓN DE PERMISOS

Se corrigieron los permisos inseguros detectados en el directorio web /var/www/html, donde durante el incidente se habían aplicado permisos excesivos como 777. Como medida de mitigación, se restauraron permisos seguros para WordPress: 755 en directorios, 644 en archivos y 640 para el archivo sensible wp-config.php. Esta corrección reduce el riesgo de modificación no autorizada de archivos, ejecución indebida y exposición de credenciales de la base de datos.

```

File Edit View Search Terminal Help
drwxr-xr-x 3 root root 4096 Sep 30 2024 ..
-rwxrwxrwx 1 www-data www-data 523 Sep 30 2024 .htaccess
-rwxrwxrwx 1 www-data www-data 10701 Sep 30 2024 index.html
-rwxrwxrwx 1 www-data www-data 405 Feb 6 2020 index.php
-rwxrwxrwx 1 www-data www-data 19903 May 12 11:05 license.txt
-rwxrwxrwx 1 www-data www-data 7425 May 12 11:05 readme.html
-rwxrwxrwx 1 www-data www-data 7349 May 12 11:05 wp-activate.php
drwxrwxrwx 9 www-data www-data 4096 Sep 10 2024 wp-admin
-rwxrwxrwx 1 www-data www-data 351 Feb 6 2020 wp-blog-header.php
-rwxrwxrwx 1 www-data www-data 2323 Jun 14 2023 wp-comments-post.php
-rwxrwxrwx 1 www-data www-data 3037 May 16 03:39 wp-config.php
-rwxrwxrwx 1 www-data www-data 3339 May 12 11:05 wp-config-sample.php
drwxrwxrwx 6 www-data www-data 4096 May 14 03:42 wp-content
-rwxrwxrwx 1 www-data www-data 5617 May 12 11:05 wp-cron.php
drwxrwxrwx 31 www-data www-data 16384 May 12 11:05 wp-includes
-rwxrwxrwx 1 www-data www-data 2493 May 12 11:05 wp-links-opml.php
-rwxrwxrwx 1 www-data www-data 3937 Mar 11 2024 wp-load.php
-rwxrwxrwx 1 www-data www-data 51437 May 12 11:05 wp-login.php
-rwxrwxrwx 1 www-data www-data 8727 May 12 11:05 wp-mail.php
-rwxrwxrwx 1 www-data www-data 31055 May 12 11:05 wp-settings.php
-rwxrwxrwx 1 www-data www-data 34516 May 12 11:05 wp-signup.php
-rwxrwxrwx 1 www-data www-data 5214 May 12 11:05 wp-trackback.php
-rwxrwxrwx 1 www-data www-data 3205 May 12 11:05 xmlrpc.php
debian@debian: /var/www/html$ █

```

Imagen 28: estado de los permisos

```

debian@debian:/var/www/html$ sudo chown -R www-data:www-data /var/www/html
[sudo] password for debian:
debian@debian:/var/www/html$ sudo find /var/www/html -type d -exec chmod 755 {}
\;
debian@debian:/var/www/html$ sudo find /var/www/html -type f -exec chmod 644 {}
\;

debian@debian:/var/www/html$ sudo chmod 640 wp-config.php
debian@debian:/var/www/html$

```

Imagen 29: corrección de los permisos 777 a permisos adecuados para directorios, archivos y wp-config.php

CORRECCIÓN DE PERMISOS Y CONTRASEÑA DEL USUARIO USER@LOCALHOST EN MARIADB

Durante el análisis forense del archivo .mysql_history se identificó la existencia del usuario user@localhost. Aunque su fecha de creación no coincide con la secuencia principal del incidente analizado el 08/10, sí se observó que mantenía una configuración insegura.

En concreto, el usuario disponía de privilegios globales sobre MariaDB mediante ALL PRIVILEGES ON *.* y además tenía habilitada la opción WITH GRANT OPTION, lo que le permitía conceder permisos a otros usuarios. Esta configuración supone un riesgo elevado y no cumple el principio de mínimo privilegio.

Además, en el histórico se observó que el usuario había sido creado con una contraseña débil y genérica. Por este motivo, aunque no se eliminó la cuenta al no poder vincularse directamente con el incidente, sí se procedió a corregir su configuración.

Primero se retiraron sus privilegios excesivos mediante la revocación de permisos globales y de la opción GRANT OPTION. Posteriormente, se cambió su contraseña por una credencial robusta y única, invalidando la contraseña anterior.

Tras aplicar la corrección, se comprobó que el usuario quedó únicamente con USAGE ON *.* , lo que indica que la cuenta sigue existiendo, pero sin permisos efectivos sobre las bases de datos. Con esta medida se reduce el riesgo de abuso de privilegios dentro de MariaDB sin eliminar una cuenta cuya relación directa con el incidente no ha podido confirmarse.

Table	Thumbnail	Summary	5 Result
Name	Keyword Preview	Location	Modified Time
wp-config.php	define('DB_USER', 'wordpressuser@localhost');	/img_debian_comprometida.E01/var/www/html/wp-c...	2024-09-30 18:02:41 CEST
mysql_history	CREATE USER 'user'@'localhost' IDENTIFIED BY 'user';	/img_debian_comprometida.E01/var/www/html/mysql...	2024-09-30 18:02:41 CEST
db.MAI	localhost	/img_debian_comprometida.E01/var/www/html/db.MAI	2024-09-30 18:02:41 CEST
wp-config.php	define('DB_USER', 'wordpressuser@localhost');	/img_debian_comprometida.E01/var/www/html/wp-c...	2024-09-30 18:02:41 CEST

Hex	Text	Application	File Metadata	OS Account	Data Artifacts	Analysis Results	Content	Annotations	Other Occurrences
String	Extracted Text	Translation							
Page 1 of 1 Page	Matches on page 1 of 2 Match	100%	Reset						
CREATE USER 'user'@'localhost' IDENTIFIED BY 'user';									
GRANT ALL PRIVILEGES ON *.* TO 'user'@'localhost' WITH GRANT OPTION;									
FLUSH PRIVILEGES;									
EXIT;									
CREATE USER 'user'@'localhost' IDENTIFIED BY 'user';									
GRANT ALL PRIVILEGES ON *.* TO 'user'@'localhost' WITH GRANT OPTION;									
FLUSH PRIVILEGES;									
EXIT;									

Imagen 30: Privilegios de usuarios de MariaDB


```
MariaDB [(none)]> SHOW GRANTS FOR 'user'@'localhost';
+-----+
| Grants for user@localhost |
+-----+
| GRANT ALL PRIVILEGES ON *.* TO `user`@`localhost` IDENTIFIED BY PASSWORD '*2470C0C06DEE42FD1618BB99005ADCA2EC9D1E19' WITH GRANT OPTION |
+-----+
```

Imagen 31: Privilegios iniciales excesivos del usuario user@localhost en MariaDB

```
MariaDB [(none)]> REVOKE ALL PRIVILEGES, GRANT OPTION FROM 'user'@'localhost';
Query OK, 0 rows affected (0.008 sec)
```

Imagen 32: Revocación de privilegios globales y GRANT OPTION al usuario user@localhost.

```
MariaDB [(none)]> ALTER USER 'user'@'localhost' IDENTIFIED BY 'Nube@82M!so1';
Query OK, 0 rows affected (0.001 sec)
```

Imagen 33: Cambio de contraseña del usuario user@localhost.

```
MariaDB [(none)]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.000 sec)

MariaDB [(none)]> SHOW GRANTS FOR 'user'@localhost;
'^> ^C
MariaDB [(none)]> SHOW GRANTS FOR 'user'@'localhost';
+-----+
| Grants for user@localhost |
+-----+
| GRANT USAGE ON *.* TO `user`@`localhost` IDENTIFIED BY PASSWORD '*BD9B9E2B2972C3AA544E9A48904688EF556F1FC4' |
+-----+
```

Imagen 34: Verificación final de permisos tras la corrección del usuario user@localhost.

CAMBIO DE CREDENCIALES EXPUESTAS Y REUTILIZADAS

En este apartado se detalla el cambio de credenciales y contraseñas inseguras y reutilizadas.

WP-CONFIG.PHP Y WORDPRESSUSER EN MARIADB

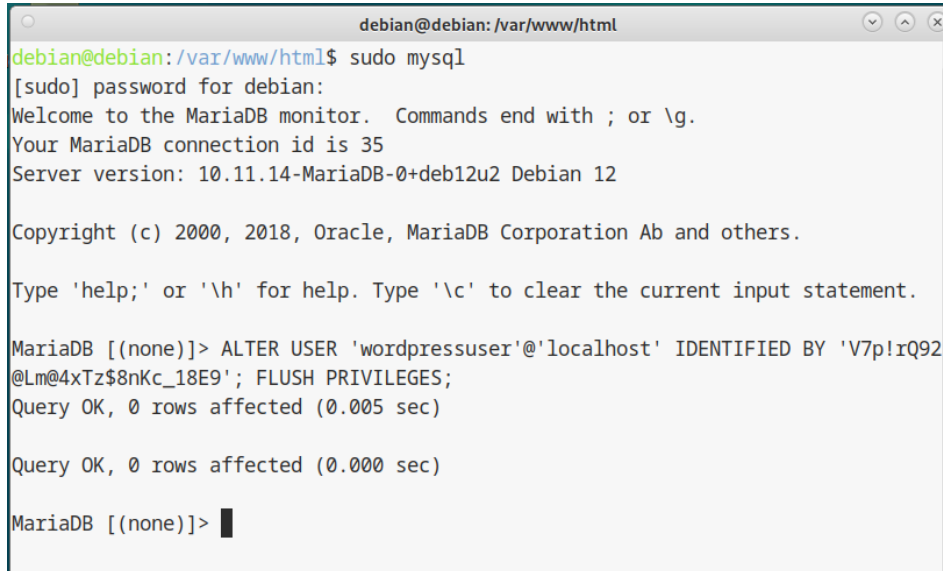
Se sustituyó la contraseña débil anterior del usuario de base de datos wordpressuser@localhost por una contraseña robusta, larga y única, compuesta por letras, mayúsculas, minúsculas, números y caracteres especiales.

Este cambio se aplicó tanto en MariaDB como en el archivo wp-config.php, ya que Wordpress utiliza las credenciales guardadas en este archivo para conectarse a la base de datos. Por este motivo, la contraseña configurada en MariaDB y la indicada en wp-config.php deben coincidir.

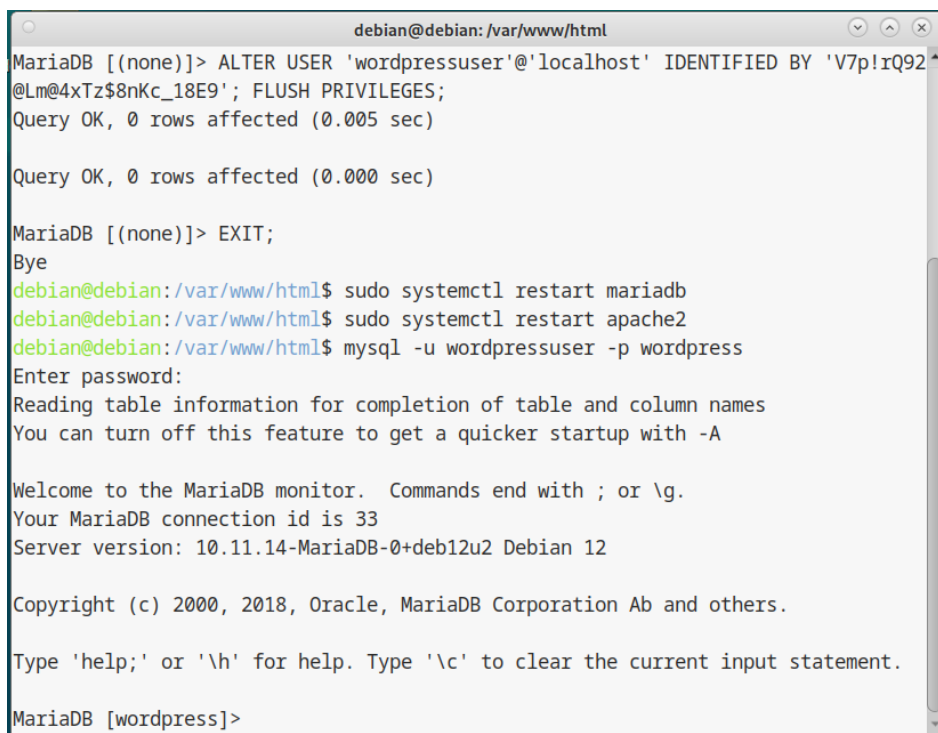
Una vez realizado el cambio, se comprobó el acceso a la base de datos con el usuario wordpressuser, confirmando que la nueva contraseña funcionaba correctamente. Con esta medida, la contraseña anterior queda invalidada y se evita su reutilización en otros usuarios o servicios del sistema.

```
/** Database password */  
define( 'DB_PASSWORD', V7p!rQ92@Lm@4xTz$8nKc_18E9 );
```

Imagen 35: Actualización de la contraseña del usuario de base de datos en wp-config.php.



```
debian@debian: /var/www/html  
debian@debian:/var/www/html$ sudo mysql  
[sudo] password for debian:  
Welcome to the MariaDB monitor.  Commands end with ; or \g.  
Your MariaDB connection id is 35  
Server version: 10.11.14-MariaDB-0+deb12u2 Debian 12  
  
Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.  
  
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.  
  
MariaDB [(none)]> ALTER USER 'wordpressuser'@'localhost' IDENTIFIED BY 'V7p!rQ92@Lm@4xTz$8nKc_18E9'; FLUSH PRIVILEGES;  
Query OK, 0 rows affected (0.005 sec)  
  
Query OK, 0 rows affected (0.000 sec)  
  
MariaDB [(none)]>
```



```
debian@debian: /var/www/html  
MariaDB [(none)]> ALTER USER 'wordpressuser'@'localhost' IDENTIFIED BY 'V7p!rQ92@Lm@4xTz$8nKc_18E9'; FLUSH PRIVILEGES;  
Query OK, 0 rows affected (0.005 sec)  
  
Query OK, 0 rows affected (0.000 sec)  
  
MariaDB [(none)]> EXIT;  
Bye  
debian@debian:/var/www/html$ sudo systemctl restart mariadb  
debian@debian:/var/www/html$ sudo systemctl restart apache2  
debian@debian:/var/www/html$ mysql -u wordpressuser -p wordpress  
Enter password:  
Reading table information for completion of table and column names  
You can turn off this feature to get a quicker startup with -A  
  
Welcome to the MariaDB monitor.  Commands end with ; or \g.  
Your MariaDB connection id is 33  
Server version: 10.11.14-MariaDB-0+deb12u2 Debian 12  
  
Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.  
  
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.  
  
MariaDB [wordpress]>
```

Imágenes 36 y 37: Cambio y validación de la nueva contraseña del usuario wordpressuser en MariaDB.

CORRECCIÓN CONTRASEÑA ADMINISTRADOR PANEL DE WORDPRESS (WORDPRESS-USER)

Durante la revisión de la base de datos de WordPress se identificaron las tablas principales de la instalación, entre ellas wp_users y wp_usermeta. A partir de la tabla wp_users se localizó el usuario wordpress-user, asociado al panel de administración de WordPress.

Posteriormente, se consultó la tabla wp_usermeta para comprobar los privilegios asociados a dicha cuenta. En el campo wp_capabilities se observó el valor administrator, confirmando que el usuario wordpress-user tenía rol de administrador dentro del panel de WordPress.

También se revisó el campo user_pass de la tabla wp_users, donde se comprobó que la contraseña no se encontraba almacenada en texto claro, sino en forma de hash. Al tratarse de una cuenta administradora dentro de un sistema comprometido se consideró necesario sustituirla por una nueva contraseña robusta y única.

Como medida de corrección, se actualizó la contraseña del usuario administrador mediante una consulta SQL sobre la tabla wp_users. Tras aplicar el cambio, se verificó que el campo user_pass había sido modificado correctamente y se comprobó el acceso al panel de administración de WordPress con la nueva credencial.

Con esta medida se invalida la contraseña anterior del administrador de WordPress y se reduce el riesgo de acceso no autorizado al panel mediante credenciales antiguas, reutilizadas o comprometidas.

```
Database changed
MariaDB [wordpress]> SHOW TABLES;
+-----+
| Tables_in_wordpress |
+-----+
| wp_commentmeta      |
| wp_comments         |
| wp_links            |
| wp_options          |
| wp_postmeta         |
| wp_posts            |
| wp_term_relationships|
| wp_term_taxonomy    |
| wp_termmeta         |
| wp_terms            |
| wp_usermeta         |
| wp_users            |
+-----+
12 rows in set (0.000 sec)
```

Imagen 38: Tablas existentes en la base de datos de Wordpress.

```
File Edit View Search Terminal Help
12 rows in set (0.000 sec)

MariaDB [wordpress]> SELECT ID, user_login, user_email, user_registered FROM wp_
users;
+-----+-----+-----+-----+
| ID | user_login | user_email | user_registered |
+-----+-----+-----+-----+
| 1 | wordpress-user | rosinnicuentas@gmail.com | 2024-09-30 16:23:12 |
+-----+-----+-----+-----+
1 row in set (0.000 sec)

MariaDB [wordpress]> SELECT u.ID, u.user_login, u.user_email, m.meta_key, m.meta_
_value FROM wp_users u JOIN wp_usermeta m ON u.ID = m.user_id WHERE m.meta_key L
IKE '%capabilities%';
+-----+-----+-----+-----+
| ID | user_login | user_email | meta_key | meta_value |
+-----+-----+-----+-----+
| 1 | wordpress-user | rosinnicuentas@gmail.com | wp_capabilities | a:1:{s:13:"
administrator";b:1;} |
+-----+-----+-----+-----+
```

Imagen 39: Identificación del usuario wordpress-user y comprobación de su rol como administrador.

```
MariaDB [wordpress]> SELECT ID, user_login, user_pass, user_email FROM wp_users;
+-----+-----+-----+-----+
| ID | user_login | user_pass | user_email |
+-----+-----+-----+-----+
| 1 | wordpress-user | $P$BM4LABXxcoawT7fuH8QRU5dcnKT2IC. | rosinnicuentas@gmail.com |
+-----+-----+-----+-----+
1 row in set (0.001 sec)

MariaDB [wordpress]>
```

Imagen 40: Hash inicial de la contraseña del usuario administrador de WordPress.

```
MariaDB [wordpress]> UPDATE wp_users SET user_pass = MD5('Luna@47R!paz') WHERE user_login = 'w
ordpress-user';
Query OK, 1 row affected (0.008 sec)
Rows matched: 1 Changed: 1 Warnings: 0
```

Imagen 41: Cambio de contraseña del usuario administrador de WordPress.

```
MariaDB [wordpress]> SELECT ID, user_login, user_pass, user_email FROM wp_users WHERE user_log
in = 'wordpress-user';
+-----+-----+-----+-----+
| ID | user_login | user_pass | user_email |
+-----+-----+-----+-----+
| 1 | wordpress-user | 17b6ecaa7c9649b3d8241a1cade43290 | rosinnicuentas@gmail.com |
+-----+-----+-----+-----+
1 row in set (0.000 sec)
```

Imagen 42: Verificación del nuevo hash tras el cambio de contraseña.

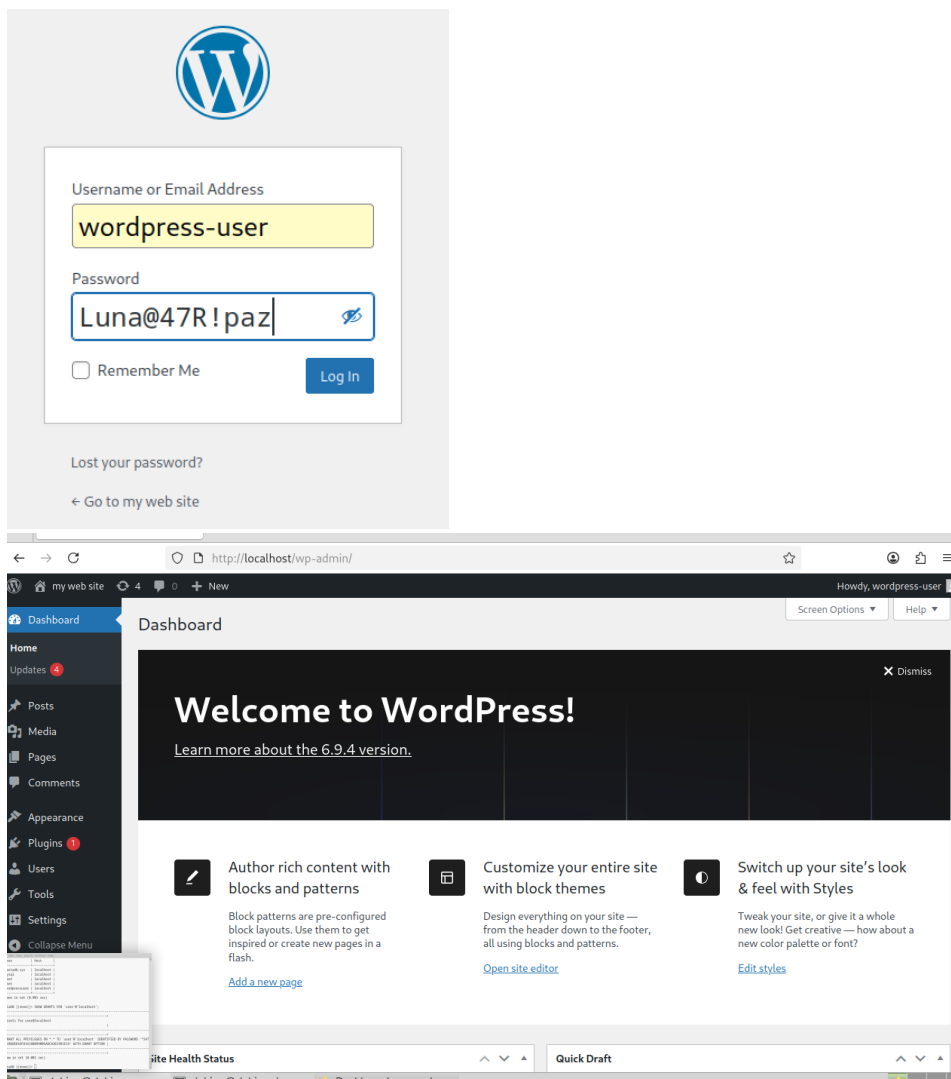


Imagen 43 y 44: Validación del acceso al panel de WordPress con la nueva credencial.

CORRECCIÓN CONTRASEÑAS DE USUARIOS DEBIAN Y ROOT

Durante el análisis del incidente se identificaron acciones administrativas realizadas desde el usuario debian y, posteriormente, un acceso SSH exitoso como root desde la dirección IP 192.168.0.134. Esta evidencia indica que las credenciales de ambos usuarios debían considerarse potencialmente comprometidas, especialmente al haberse permitido el acceso por SSH mediante contraseña y el inicio de sesión directo como root.

Como medida de corrección, se procedió a cambiar las contraseñas de los usuarios debian y root, sustituyéndolas por credenciales robustas, únicas y no reutilizadas en otros servicios. Esta medida permite invalidar las contraseñas anteriores y reducir el riesgo de que puedan volver a utilizarse para acceder al sistema.

Tras ejecutar el cambio, el sistema confirmó la actualización correcta de la contraseñas mediante el mensaje password updated successfully. Se documentan las nuevas contraseñas por tratarse de un informe con fines educativos, y por la necesidad de la academia de comprobar las correcciones realizadas a la máquina comprometida. En un escenario real, o se documentarían por motivos de seguridad.

Con esta corrección se refuerza la seguridad de las cuentas locales del sistema y se reduce la posibilidad de nuevos accesos no autorizados mediante credenciales anteriores, débiles o comprometidas.

Nueva contraseña debian: Faro@39M!pez

Nueva contraseña root: Lago@58N!mar

```
debian@debian:/$ sudo passwd debian
[sudo] password for debian:
New password:
Retype new password:
passwd: password updated successfully
debian@debian:/$
debian@debian:/$ sudo passwd root
New password:
Retype new password:
passwd: password updated successfully
debian@debian:/$
```

Imágenes 45 y 46: cambio exitoso de contraseñas de usuarios debian y root.

CORRECCIÓN DE LA CONFIGURACIÓN DE APACHE

Para corregir la configuración insegura de Apache, se modificó el archivo `/etc/apache2/apache2.conf`. Durante el análisis se había identificado que el bloque `<Directory />`, que hace referencia a la raíz del sistema, tenía configurado `Require all granted`. Esta configuración era demasiado permisiva, ya que dejaba permitido el acceso por defecto a una ruta que no debería estar accesible desde el servicio web.

Como medida de corrección, se cambió esta directiva por `Require all denied`, dejando denegado el acceso desde la raíz del sistema. De esta forma, Apache solo podrá servir contenido desde los directorios que estén permitidos de forma explícita.

También se quitó la opción `Indexes`, ya que esta permitía que apache mostrara el listado de archivos de un directorio cuando no había un archivo índice. En su lugar, se dejó la configuración `Options -Indexes +FollowSymLinks`, permitiendo que la web siguiera funcionando, pero evitando que se pudieran ver los archivos de carpetas como `wp-content/uploads`.

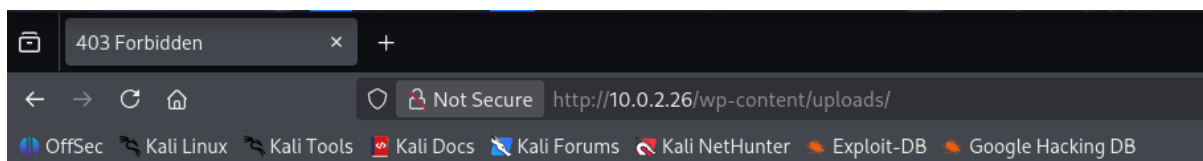
Después de aplicar los cambios, se comprobó desde el navegador que al acceder directamente al directorio `wp-content/uploads/` ya no se mostraba el listado de archivos, sino un mensaje `Forbidden`. Confirmando que el listado de directorios quedó deshabilitado correctamente.

```
GNU nano 7.2 /etc/apache2/apache2.conf *
# your system is serving content from a sub-directory in /srv you must allow
# access here, or in any related virtual host.
<Directory />
    Options FollowSymLinks
    AllowOverride None
    Require all denied
</Directory>

<Directory /usr/share>
    AllowOverride None
    Options -Indexes +FollowSymLinks
    Require all granted
</Directory>

<Directory /var/www/>
    Options -Indexes +FollowSymLinks
    AllowOverride None
    Require all granted
</Directory>
```

Imagen 47: Configuración corregida de Apache, con acceso denegado por defecto en <Directory /> y listado de directorios deshabilitado



Forbidden

You don't have permission to access this resource.

Apache/2.4.67 (Debian) Server at 10.0.2.26 Port 80

Imagen 48: Comprobación desde el navegador de que el directorio ya no muestra listado de archivos y devuelve Forbidden.

RECOMENDACIONES

Además de las medidas concretas de contención y corrección del incidente, se proponen una serie de recomendaciones generales para reducir la probabilidad de que se produzcan ataques similares en el futuro.

GESTIÓN SEGURA DE ACCESOS

Se recomienda establecer una política clara de gestión de accesos al servidor. Los usuarios deben tener permisos acordes a sus funciones y no deben utilizarse cuentas compartidas o cuentas privilegiadas para tareas habituales.

También se recomienda revisar periódicamente qué usuarios tienen acceso al sistema, qué permisos tienen asignados y si siguen necesitando dicho acceso.

También se recomienda revocar las cuentas de empleados o usuarios que ya no formen parte de la organización.

PRINCIPIO DE MÍNIMO PRIVILEGIO

Todos los servicios, usuarios y directorios deben configurarse siguiendo el principio de mínimo privilegio, para que cada cuenta o servicio tenga únicamente los permisos necesarios para funcionar.

Aplicar este principio reduce el impacto de un posible compromiso, ya que aunque un atacante consiga acceso a una cuenta, sus posibilidades de modificar el sistema o escalar privilegios serán más limitadas.

POLÍTICA DE CONTRASEÑAS Y AUTENTICACIÓN

Se recomienda encarecidamente implantar una política de contraseñas seguras, evitando contraseñas débiles, predecibles, repetidas o compartidas entre varios servicios. Las contraseñas deben ser robustas, únicas y revisarse de forma periódica.

También se debe complementar la autenticación con métodos más seguros como autenticación multifactor, o claves en servicios de administración remota como SSH.

REDUCCIÓN DE LA SUPERFICIE DE EXPOSICIÓN

El servidor debe mantener activos únicamente los servicios necesarios para su funcionamiento. Cualquier servicio que no sea imprescindible aumenta la superficie de ataque y puede convertirse en una vía de entrada.

Por este motivo, se recomienda revisar periódicamente los puertos abiertos, los servicios instalados y las aplicaciones expuestas, eliminando o deshabilitando aquello que no sea necesario.

HARDENING DE SERVICIOS CRÍTICOS

Los servicios críticos, como SSH, Apache, FTP, bases de datos y WordPress, deben configurarse siguiendo las buenas prácticas de seguridad.

Esto incluye evitar configuraciones por defecto inseguras, limitar accesos innecesarios, revisar permisos, desactivar funcionalidades no utilizadas y comprobar que los servicios no permitan más acceso del necesario.

CIFRADO DE INFORMACIÓN SENSIBLE

Se recomienda aplicar cifrado sobre la información sensible del servidor, especialmente credenciales, copias de seguridad, bases de datos y archivos de configuración críticos. En el caso de WordPress, archivos como wp-config.php contienen datos sensibles, por lo que deben protegerse mediante permisos restrictivos y, cuando sea posible, mediante mecanismos adicionales de cifrado o protección de secretos.

También se recomienda cifrar las copias de seguridad antes de almacenarlas o transferirlas, para evitar que un atacante pueda acceder a información crítica, aunque consiga copiar los archivos.

MONITORIZACIÓN CON SIEM Y ALERTAS

Se recomienda implantar una solución SIEM, como Wazuh, para centralizar los registros del sistema y detectar actividad sospechosa. El SIEM debería recoger logs de SSH, Apache, FTP, WordPress, sistema operativo y base de datos, generando alertas ante eventos relevantes como accesos administrativos, múltiples intentos fallidos, cambios en archivos críticos, instalación de nuevos servicios o modificaciones de permisos.

Además, se recomienda revisar periódicamente las alertas y paneles del SIEM para comprobar que las reglas funcionan correctamente, detectar falsos positivos, identificar patrones anómalos y ajustar la monitorización según evolucione el entorno.

ACTUALIZACIÓN Y MANTENIMIENTO

Se recomienda mantener actualizado el sistema operativo, los paquetes instalados, WordPress, plugins y temas, para que un atacante no pueda aprovechar vulnerabilidades conocidas para acceder al sistema.

También es importante eliminar plugins, temas o servicios que no se utilicen, ya que pueden quedar desactualizados y convertirse en puntos débiles.

COPIAS DE SEGURIDAD Y RECUPERACIÓN

Se debe contar con una política de copias de seguridad periódicas del sistema, la base de datos y los archivos web. Las copias deben almacenarse en una ubicación separada y probarse regularmente para asegurar que se pueden restaurar los datos correctamente.

Esto permite recuperar el servicio con rapidez en caso de modificación o eliminación de archivos ya sea de forma accidental o maliciosa.

CONTROL DE CAMBIOS

Cualquier modificación en servicios críticos debe quedar documentada. Es recomendable registrar qué se hizo, quién lo hizo, cuándo y por qué motivo.

Este control ayuda a diferenciar actividad administrativa legítima de actividad sospechosa, facilitando futuras investigaciones en caso de un nuevo incidente

FORMACIÓN Y CONCIENCIACIÓN

Por último, se recomienda formar a todo el equipo en ciberseguridad y buenas prácticas, ya que el factor humano suele ser una vía de entrada común a los sistemas.

La concienciación ayuda a prevenir errores humanos y mejora la capacidad de respuesta ante señales de actividad sospechosa.